

Model Validation, Overfitting Control & Hyperparameter Tuning

1. Overfitting Analysis

Introduction

The objective of this project was to evaluate and improve the performance of machine learning regression models using the California Housing dataset. The dataset contains housing-related attributes such as median income, house age, average rooms, population, and location coordinates. The target variable represents the median house value.

Three regression models were considered:

- Linear Regression
- Ridge Regression
- Decision Tree Regressor

The dataset was first split into training and testing sets using an 80:20 ratio. Feature scaling was applied using StandardScaler to ensure that numerical features were standardized before training the models.

Understanding Overfitting

Overfitting occurs when a machine learning model learns the training data too well, including noise and minor fluctuations. As a result, the model performs exceptionally on training data but performs poorly on unseen testing data.

To identify overfitting, a Decision Tree Regressor was trained and evaluated on both the training and testing datasets.

The training and testing Root Mean Squared Error (RMSE) values were compared. A large difference between training and testing performance indicates that the model memorized the training dataset rather than learning general patterns.

Overfitting Observation

The baseline Decision Tree model produced a significantly lower training error compared to testing error. This behavior suggests that the model was overly complex and captured dataset-specific patterns.

Key indicators of overfitting included:

- Very strong performance on training data.
- Reduced performance on testing data.
- Noticeable difference between training and testing RMSE values.

Therefore, additional validation and tuning techniques were required to improve generalization performance.

2. Hyperparameter Tuning Approach

Cross-Validation

To obtain a more reliable estimate of model performance, 5-fold cross-validation was applied.

Cross-validation divides the dataset into five subsets (folds). During each iteration:

1. Four folds are used for training.
2. One fold is used for validation.
3. The process repeats five times.
4. The average score is calculated.

Benefits of cross-validation include:

- More reliable model evaluation.
- Reduced dependence on a single train-test split.
- Better estimation of real-world performance.

The average cross-validation RMSE provided a stable measure of model quality and helped verify whether the model was overfitting.

Hyperparameter Tuning using GridSearchCV

To improve the Decision Tree model, GridSearchCV was used.

GridSearchCV systematically tests different combinations of hyperparameters and selects the combination that produces the best cross-validation score.

The following hyperparameters were tuned:

- max_depth
- min_samples_split

Example parameter grid:

```
param_grid = {  
    "max_depth": [3, 5, 7, 10],  
    "min_samples_split": [2, 5, 10]  
}
```

GridSearchCV evaluated each parameter combination using 5-fold cross-validation and identified the optimal configuration.

Benefits of Hyperparameter Tuning

The tuning process helped:

- Reduce overfitting.
- Improve generalization capability.

- Increase prediction accuracy.
- Select the most effective model configuration automatically.

The best parameter combination returned by GridSearchCV was used to train the final optimized Decision Tree model.

3. Final Model Justification

Model Evaluation Metrics

The models were evaluated using two primary metrics:

Root Mean Squared Error (RMSE)

RMSE measures the average prediction error of the model.

A lower RMSE indicates:

- Better prediction accuracy.
- Smaller prediction errors.

R² Score

R² measures how well the model explains variation in the target variable.

An R² score closer to 1 indicates stronger predictive performance.

Model Comparison

Three models were compared:

1. Linear Regression
2. Ridge Regression
3. Tuned Decision Tree

The comparison was performed using:

- RMSE
- R² Score

The model comparison table clearly demonstrated the strengths and weaknesses of each model.

Linear Regression provided a simple baseline model and produced reasonable results.

Ridge Regression introduced regularization and helped control model complexity, producing performance similar to or slightly better than Linear Regression.

The Tuned Decision Tree model benefited from hyperparameter optimization and demonstrated improved predictive capability.

Final Model Selection

The final model was selected based on the following criteria:

- Lowest RMSE.
- Highest R^2 score.
- Better generalization performance.
- Reduced overfitting after tuning.

Based on these evaluation metrics, the Tuned Decision Tree model was chosen as the final model because it achieved the best balance between accuracy and generalization.

Conclusion

This project demonstrated the importance of model validation and hyperparameter tuning in machine learning. Cross-validation was used to obtain reliable performance estimates, while GridSearchCV helped identify the optimal Decision Tree configuration.

The tuned model achieved improved predictive performance compared to the baseline models and was therefore selected as the final solution. The project highlights how proper validation techniques can reduce overfitting and improve the reliability of machine learning systems.